

GPU Accelerated L-System for Botanical Structures

Eric Lee, CS 179 Project

May 7, 2026

1 Background

In 1968, Aristid Lindenmayer introduced L-systems¹ to model the growth and development of multicellular filamentous organisms, such as the growth of algae and the branching architecture of higher plants. Unlike sequential rewriting systems, L-systems apply production rules in parallel to every symbol at each timestep, mirroring biological cell division in which daughter cells mature simultaneously. This formalism captures self-similar morphogenesis, simple deterministic rules generate intricate filaments, branching structures, and fractal curves that closely resemble natural botanical forms.

Beyond biology, L-systems generalize to classical space-filling fractals such as the Koch curve and Sierpiński triangle. They also extend to bracketed alphabets² that encode nested branching by pushing and popping a turtle’s geometric state. Despite their simplicity, efficient evaluation is computationally demanding because string length grows exponentially with the iteration count, and bracketed structures introduce sequential dependencies making naive parallelization not possible.

2 L-Systems

Formally, an L-system is a triple $G = (V, \omega, P)$, where V is a finite alphabet of symbols, $\omega \in V^+$ is the axiom (the initial string), and $P \subseteq V \times V^*$ is a set of production rules of the form $a \rightarrow \chi$, mapping each symbol to a replacement string. Starting from ω , the system generates a sequence of strings $\sigma_0, \sigma_1, \sigma_2, \dots$ by simultaneously rewriting every symbol of σ_n according to P to obtain σ_{n+1} . This simultaneous replacement is what distinguishes an L-system from a Chomsky grammar and is what makes derivation parallel.

To produce branching botanical forms, the alphabet is augmented with bracket symbols $[$ and $]$ that respectively push and pop the state of a turtle interpreter $(\mathbf{p}, \mathbf{R}) \in \mathbb{R}^3 \times SO(3)$.

¹Przemysław Prusinkiewicz and Aristid Lindenmayer. *The Algorithmic Beauty of Plants*. New York: Springer-Verlag, 1990.

²Seymour Ginsburg and Michael A. Harrison. “Bracketed context-free languages”. In: *Journal of Computer and System Sciences* 1.1 (1967), pp. 1–23. DOI: 10.1016/S0022-0000(67)80003-5. URL: [https://doi.org/10.1016/S0022-0000\(67\)80003-5](https://doi.org/10.1016/S0022-0000(67)80003-5).

The standard turtle commands are:

- **F**: move forward and draw a segment,
- **+ / -**: rotate by a fixed angle δ ,
- **[/]**: push and pop the turtle state, beginning or ending a branch.

As a concrete example, consider the bracketed system for a fractal plant. The axiom is $\omega = -X$ and the production rules are

$$\begin{aligned} X &\rightarrow F + [[X] - X] - F [-FX] + X, \\ F &\rightarrow FF, \end{aligned}$$

interpreted with rotation angle $\delta = 25^\circ$. Here, **X** does not correspond to any drawing action, it just controls the structural evolution of the curve. The first two derivations are:

$$\begin{aligned} \sigma_0 &: -X, \\ \sigma_1 &: -F + [[X] - X] - F [-FX] + X. \end{aligned}$$

Each **X** is replaced in parallel by the same branch-producing pattern wrapped with nested brackets, while every **F** simultaneously doubles. Consequently each iteration both lengthens existing segments and grafts new sub-branches throughout the structure. After only a few iterations, the turtle traces out a plant-like figure.



Figure 1: Barnsley Fern, $\sigma = 6$

The string length grows exponentially in the iteration count (here, 3^n symbols after n rewrites of an **F**-only axiom under this rule), which is exactly what makes evaluation expensive on a CPU and attractive on a GPU.

3 Parallel GPU Algorithm

We propose a hybrid CPU–GPU pipeline. The CPU parses the grammar, sequences iterations, and provides a sequential fallback for the bracketed branch-resolution stage. The GPU performs bulk string expansion, accumulates affine transforms through a parallel prefix scan, and writes geometry directly into OpenGL VBOs through CUDA. A sequential CPU implementation provides the correctness baseline and CPU timing data.

The primary issue is the variable-length expansion of symbols. A single F may rewrite to many commands, and every symbol is replaced simultaneously, so threads cannot serially append to a shared buffer. We decompose one rewrite step into three phases:

1. A length kernel assigns one thread per input symbol and records its production’s output length in an array L .
2. An exclusive prefix sum $S = \text{exclusive_scan}(L)$ yields each thread’s write offset.
3. A scatter kernel writes the expanded tokens into the output buffer in parallel using offsets from S .

Two preallocated ping-pong device buffers avoid repeated allocations, so iterating n times produces σ_n entirely on-device. For non-bracketed systems (Koch, Dragon, Sierpiński), the world transform of the i -th segment is $T_i = \prod_{j=0}^{i-1} M_j$. Since matrix multiplication is associative, this reduces to a parallel exclusive scan over 4×4 affine matrices.

Bracketed systems introduce a dependency tree, as a branch must restore the transform stored at its matching $[$, which breaks the simple linear scan and rules out per-thread stacks (which would cause severe warp divergence and register pressure). Therefore, we adopt a hybrid strategy: the GPU still expands the string in parallel, but a lightweight CPU turtle interpreter resolves the brackets and writes vertices into the mapped VBO. This guarantees a working demo for fractal plants while still placing the asymptotically dominant work (string expansion, which grows exponentially) on the GPU.

Rendering will be done through CUDA/OpenGL interop, where compute kernels write instance attributes (position, orientation, length, color) into a buffer that OpenGL consumes via instanced drawing, with orbit camera, anti-aliased lines, and optional bloom handled by the graphics pipeline.

4 Deliverables and Goals

The core work consists of a CPU proof-of-concept rewriter, a GPU string expander, a GPU transform engine that uses parallel matrix prefix scan for non-bracketed curves, and interactive OpenGL rasterizer with orbit camera with live parameter editing with the CUDA OpenGL interop. A performance pass will produce GPU and CPU speedup plots and Nsight Compute occupancy and bandwidth metrics.

There are some stretch goals if we finish early. First is a full GPU bracket reducer replacing the CPU interpreter via scan-based bracket matching and per-branch tree reduc-

tion. Another is CUDA Kawase bloom post-processing and other visualizations. Finally, it would be interesting to render fragment shaders in GLML (<https://www.glml-lang.com/>), a functional DSL compiling to GLSL written by the author to learn how to integrate a custom shader compiler with the CUDA/OpenGL pipeline.

5 Proposed Timeline

- Week 1 (CPU baseline): Grammar parser, sequential rewriter, and turtle interpreter for both non-bracketed and bracketed systems, CPU timing data on various examples
- Week 2 (GPU string expansion): Kernels, validate against CPU output
- Week 3 (GPU rendering and CPU demo): Parallel matrix-scan for non-bracketed transform accumulation, OpenGL renderer with orbit camera, render fractal plant examples
- Week 4 (Profiling): Nsight Compute optimization, CPU vs. GPU comparisons, attempt the GPU bracket reducer if time permits, write-up

References

- Ginsburg, Seymour and Michael A. Harrison. “Bracketed context-free languages”. In: *Journal of Computer and System Sciences* 1.1 (1967), pp. 1–23. DOI: 10.1016/S0022-0000(67)80003-5. URL: [https://doi.org/10.1016/S0022-0000\(67\)80003-5](https://doi.org/10.1016/S0022-0000(67)80003-5).
- Prusinkiewicz, Przemyslaw and Aristid Lindenmayer. *The Algorithmic Beauty of Plants*. New York: Springer-Verlag, 1990.